

Competitive Programming

Section 42 Editorial: Advanced Data Structures Mixed Contest

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	3
1.1	Detailed solution guide	3
2	A. Dynamic Order Statistics	4
2.1	Restatement	4
2.2	Observations and algorithm	4
2.3	Correctness proof	4
2.4	Complexity	4
2.5	Full C++ solution	4
2.6	Full Python solution	4
2.7	Tests that matter	5
3	B. Range Add Range Sum	6
3.1	Restatement	6
3.2	Observations and algorithm	6
3.3	Correctness proof	6
3.4	Complexity	6
3.5	Full C++ solution	6
3.6	Full Python solution	6
3.7	Tests that matter	7
4	C. Static Rectangle Count	8
4.1	Restatement	8
4.2	Observations and algorithm	8
4.3	Correctness proof	8
4.4	Complexity	8
4.5	Full C++ solution	8
4.6	Full Python solution	8
4.7	Tests that matter	9
5	What to carry forward	10

1 Editorial overview

1.1 Detailed solution guide

A. Dynamic Order Statistics. A Fenwick tree over value coordinates stores frequencies. $\text{sum}(x)$ is the number of elements $\leq x$. Insert and erase update one coordinate by $+1$ or -1 . The k -th element is the smallest coordinate whose prefix sum is at least k ; binary lifting inside the Fenwick tree finds that coordinate in logarithmic time.

B. Range Add, Range Sum. Difference arrays turn a range add into two point events, but range-sum queries need one more accumulated moment. Maintain two Fenwick trees $B1$ and $B2$ so that after adding delta to $[l, r]$, $\text{prefix}(p) = p \cdot \text{sum}(B1, p) - \text{sum}(B2, p)$. Then $\text{sum}(l, r)$ is $\text{prefix}(r) - \text{prefix}(l-1)$.

C. Offline Rectangle Counting. Count points in rectangle $[x1, x2] \times [y1, y2]$ by inclusion-exclusion of four dominance queries $\text{count}(x \leq X, y \leq Y)$. Sort points and queries by X , sweep from left to right, and insert active point y -coordinates into a Fenwick tree. The y -prefix sum then answers the dominance query.

Common pitfalls. Coordinate-compress values before using a Fenwick tree when coordinates are large. For range sums, update both trees with matching signs. For rectangles, inclusion-exclusion signs are $+$ $-$ $-$ $+$.

All three problems reduce a seemingly richer query to prefix queries.

- A stores frequencies in a Fenwick tree. Counts are prefix sums; k -th order statistic is the first index whose prefix sum reaches k .
- B stores a range-added array through two Fenwick trees. The formula $\text{prefix}(p) = p \cdot \text{sum}(B1, p) - \text{sum}(B2, p)$ converts range additions into prefix sums of the final array.
- C expands each rectangle into four dominance queries $x \leq X, y \leq Y$, then sweeps points by x -coordinate and answers y -prefix counts with a Fenwick tree.

The recurring invariant is that the Fenwick tree contains exactly the active objects for the current prefix threshold.

2 A. Dynamic Order Statistics

2.1 Restatement

Maintain a multiset over values $1..m$: insert, erase one copy, k-th, and count $\leq x$.

2.2 Observations and algorithm

Use the detailed solution guide above as the roadmap. The implementation below follows the stated invariant directly and handles the edge cases called out for this exercise.

2.3 Correctness proof

Correctness follows from the problem-specific argument in the detailed guide: the maintained structure represents exactly the objects claimed there, and each answer is read from that representation. $\square$

2.4 Complexity

The running time is the one derived in the detailed guide and visible in the loops below: pre-processing builds the structure once, while each operation performs the stated scans, searches, jumps, or graph traversals.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; struct BIT{int n; vector<int> b; BIT(int n):n(n),b(n+1){} void add(int
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int m,q;if(!(cin>m>q)) return 0; E
```

2.6 Full Python solution

```

import sys
class BIT:
    def __init__(self,n): self.n=n; self.bit=[0]*(n+1)
    def add(self,i,x):
        while i<=self.n: self.bit[i]+=x; i+=i&-i
    def sum(self,i):
        s=0
        while i>0: s+=self.bit[i]; i-=i&-i
        return s
    def kth(self,k):
        if k<1 or k>self.sum(self.n): return -1
        pos=0; step=1<<self.n.bit_length()
        while step:
            nxt=pos+step
            if nxt<=self.n and self.bit[nxt]<k: pos=nxt; k-=self.bit[nxt]
            step//=2
        return pos+1
def main():
    data=list(map(int,sys.stdin.buffer.read().split()));
    if not data: return
    m,q=data[0],data[1]; bit=BIT(m); cnt=[0]*(m+1); idx=2; out=[]
    for _ in range(q):
        t,x=data[idx],data[idx+1]; idx+=2
        if t==1: cnt[x]+=1; bit.add(x,1)
        elif t==2:
            if cnt[x]: cnt[x]-=1; bit.add(x,-1)
        elif t==3: out.append(str(bit.kth(x)))
        else: out.append(str(bit.sum(x)))
    print("\n".join(out))
if __name__=="__main__": main()

```

2.7 Tests that matter

Use singleton inputs, repeated values, empty answers, ancestor/self paths, disconnected matching edges, and bottleneck flow graphs.

3 B. Range Add Range Sum

3.1 Restatement

Support range addition and range-sum queries.

3.2 Observations and algorithm

Use the detailed solution guide above as the roadmap. The implementation below follows the stated invariant directly and handles the edge cases called out for this exercise.

3.3 Correctness proof

Correctness follows from the problem-specific argument in the detailed guide: the maintained structure represents exactly the objects claimed there, and each answer is read from that representation. $\square$

3.4 Complexity

The running time is the one derived in the detailed guide and visible in the loops below: pre-processing builds the structure once, while each operation performs the stated scans, searches, jumps, or graph traversals.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; struct BIT{int n; vector<long long>b; BIT(int n):n(n),b(n+2){} void add
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,q;if(!(cin>n>q)) return 0; E
```

3.6 Full Python solution

```

import sys
class BIT:
    def __init__(self,n): self.n=n; self.bit=[0]*(n+2)
    def add(self,i,x):
        while i<=self.n: self.bit[i]+=x; i+=i&-i
    def sum(self,i):
        s=0
        while i>0: s+=self.bit[i]; i-=i&-i
        return s
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,q=data[0],data[1]; a=data[2:2+n]; b1=BIT(n+1); b2=BIT(n+1)
    def add_range(l,r,x): b1.add(l,x); b1.add(r+1,-x); b2.add(l,x*(1-1)); b2.add(r+1,-x*r)
    def pref(i): return b1.sum(i)*i-b2.sum(i)
    for i,x in enumerate(a,1): add_range(i,i,x)
    idx=2+n; out=[]
    for _ in range(q):
        t=data[idx]; idx+=1
        if t==1:
            l,r,x=data[idx],data[idx+1],data[idx+2]; idx+=3; add_range(l,r,x)
        else:
            l,r=data[idx],data[idx+1]; idx+=2; out.append(str(pref(r)-pref(l-1)))
    print("\n".join(out))
if __name__=="__main__": main()

```

3.7 Tests that matter

Use singleton inputs, repeated values, empty answers, ancestor/self paths, disconnected matching edges, and bottleneck flow graphs.

4 C. Static Rectangle Count

4.1 Restatement

Given points and rectangle queries, count points inside each rectangle.

4.2 Observations and algorithm

Use the detailed solution guide above as the roadmap. The implementation below follows the stated invariant directly and handles the edge cases called out for this exercise.

4.3 Correctness proof

Correctness follows from the problem-specific argument in the detailed guide: the maintained structure represents exactly the objects claimed there, and each answer is read from that representation. $\square$

4.4 Complexity

The running time is the one derived in the detailed guide and visible in the loops below: pre-processing builds the structure once, while each operation performs the stated scans, searches, jumps, or graph traversals.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; struct BIT{int n; vector<int>b; BIT(int n):n(n),b(n+1){} void add(int i, int v){for(;i<=n;i+=i&-i)b[i]+=v;} int sum(int i){int s=0; for(;i>0;i-=i&-i)s+=b[i]; return s;} };
struct E{int x,y1,y2,id,sgn; bool operator<(const E&o)const{return x<o.x;}};
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,q;if(!(cin>>n>>q)) return 0; v
```

4.6 Full Python solution

```

import sys,bisect
class BIT:
    def __init__(self,n): self.n=n; self.bit=[0]*(n+1)
    def add(self,i,x):
        while i<=self.n: self.bit[i]+=x; i+=i&-i
    def sum(self,i):
        s=0
        while i>0: s+=self.bit[i]; i-=i&-i
        return s
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,q=data[0],data[1]; idx=2; pts=[]; ys=[]
    for _ in range(n): x,y=data[idx],data[idx+1]; idx+=2; pts.append((x,y)); ys.append(y)
    events=[]; ans=[0]*q
    for qi in range(q):
        x1,y1,x2,y2=data[idx],data[idx+1],data[idx+2],data[idx+3]; idx+=4; ys+= [y1,y2]
        events.append((x2,y1,y2,qi,1)); events.append((x1-1,y1,y2,qi,-1))
    ys=sorted(set(ys)); pts.sort(); events.sort(); bit=BIT(len(ys)); p=0
    for x,y1,y2,qi,sgn in events:
        while p<n and pts[p][0]<=x:
            bit.add(bisect.bisect_left(ys,pts[p][1])+1,1); p+=1
        l=bisect.bisect_left(ys,y1)+1; r=bisect.bisect_right(ys,y2)
        ans[qi]+=sgn*(bit.sum(r)-bit.sum(l-1))
    print("\n".join(map(str,ans)))
if __name__=="__main__": main()

```

4.7 Tests that matter

Use singleton inputs, repeated values, empty answers, ancestor/self paths, disconnected matching edges, and bottleneck flow graphs.

5 What to carry forward

Advanced algorithms become manageable when you state the invariant first, then code only the operations needed to preserve it.